

Shallow Copy and Deep Copy in C# with Examples

In this article, I will discuss the **Shallow Copy and Deep Copy in C#** with Examples. Please read our previous article discussing [Prototype Design Patterns in C#](#) with Examples. Shallow Copy and Deep Copy play important roles in copying the objects in the Prototype Design Pattern. So, in this article, we will discuss Shallow Copy and Deep Copy and the differences between them with examples.

Understanding Shallow Copy in C#:

In the case of Shallow copy, it will create the new object from the existing object and then copy the value type fields of the current object to the new object. But in the case of reference type, it will only copy the reference, not the referred object itself. Therefore, the original and clone refer to the same object in the case of reference type.

- **Value Types:** Each value-type field is copied directly. Value types (such as integers and structs) store data directly, so a direct value copy is made.
- **Reference Types:** For reference types fields, the reference is copied but not the underlying object. Thus, the original object and its copy will refer to the same object.

To understand this better, please have a look at the following diagram. As you can see in the below image, we are having Employee and Address classes. Further, you notice inside the Employee class, we have declared two value-type properties, i.e., Name and Department, and one reference-type property, i.e., EmpAddress. In the case of Shallow Copy, the value of the value type properties will be cloned while the reference address of the reference type will be cloned into the new object. With Shall Copy in C#, both existing and cloned objects will point to the same memory location in the case of reference type property.

```
public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public Address EmpAddress { get; set; }

    public Employee GetClone()
    {
        return (Employee)this.MemberwiseClone();
    }
}

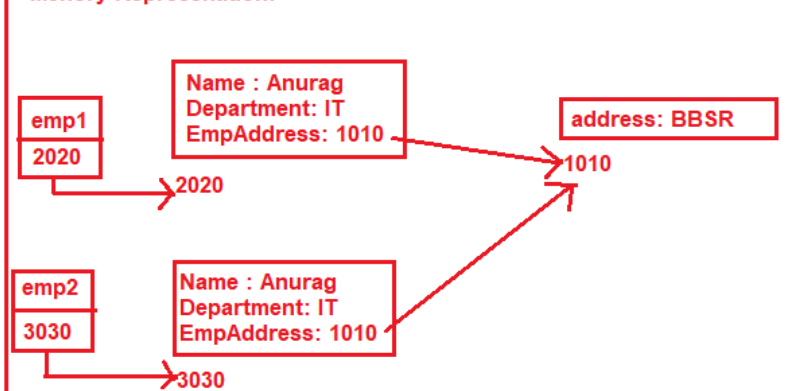
public class Address
{
    public string address { get; set; }
}
```

```
static void Main(string[] args)
{
    Employee emp1 = new Employee();
    emp1.Name = "Anurag";
    emp1.Department = "IT";
    emp1.EmpAddress = new Address() { address = "BBSR" };

    Employee emp2 = emp1.GetClone();
    emp2.Name = "Pranaya";
    emp2.EmpAddress.address = "Mumbai";
}
```

Client Code

Memory Representation:



As shown in the above diagram, first, we create an object, i.e., emp1, and then initialize the object with some values. Then, we create the second object, i.e., emp2, using the GetClone method. The memory representation shows that the value type fields (Name and Department) are copied and stored in a different memory location. In contrast, the reference type field, i.e., EmpAddress, still points to the same old memory location. That means both emp1 and emp2 now refer to the same Address object. So, if we make any changes to the employee address, they will affect each other.

Example to Understand Shallow Copy in C#:

The following Example shows the use of Shallow Copy in C#. The following Example code is self-explained, so please go through the comment lines for a better understanding.

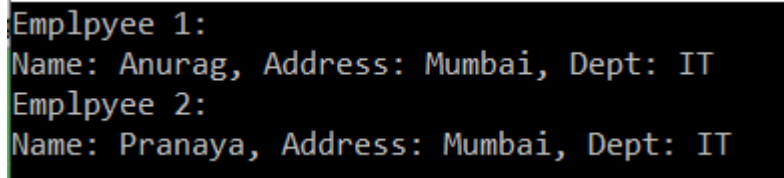
```
using System;
namespace ShallowCopyDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Example to Understand Shallow Copy
            //Creating Employee Object
            Employee emp1 = new Employee
            {
                Name = "Anurag",
                Department = "IT",
                EmpAddress = new Address() { address = "BBSR" }
            };
            //Creating a Clone Object from the Existing Object
            Employee emp2 = emp1.GetClone();
            //Changing Name Property of Clone Object Will Not Reflect the
Existing Object
            emp2.Name = "Pranaya";
            //Changing Address Property of Clone Object Will Reflect the
Changes of the Existing Object
            //This is because address is a reference type property and in
the case of Shallow Copy
            //Both Clone and Existing Object will point to the Same Memory
Address
            emp2.EmpAddress.address = "Mumbai";
            Console.WriteLine("Employee 1: ");
            Console.WriteLine("Name: " + emp1.Name + ", Address: " +
emp1.EmpAddress.address + ", Dept: " + emp1.Department);
            Console.WriteLine("Employee 2: ");
            Console.WriteLine("Name: " + emp2.Name + ", Address: " +
emp2.EmpAddress.address + ", Dept: " + emp2.Department);
            Console.Read();
        }
    }
    public class Employee
    {
        //Value type Property
        public string Name { get; set; }
        //Value type Property
        public string Department { get; set; }
        //Reference type Property
        public Address EmpAddress { get; set; }
        //Creating a Cloned Object of the Current Object
        public Employee GetClone()
        {
            //Both Cloned and Existing Object Point to the Same Memory
Location of the Address Object
```

```

        return (Employee)this.MemberwiseClone();
    }
}
public class Address
{
    public string address { get; set; }
}
}

```

In the above code, after creating the second employee object, the cloned employee, we change the address and name of the second employee, emp2. Now, when you run the application, you will see that the address is also changed for emp1, as shown in the image below.



```

Employee 1:
Name: Anurag, Address: Mumbai, Dept: IT
Employee 2:
Name: Pranaya, Address: Mumbai, Dept: IT

```

Note: In C#, the term “shallow copy” refers to a method of copying an object where only the references to the objects are copied and not the actual objects themselves. If the original object contains references to other objects (like arrays or class objects), then in a shallow copy, both the original and the copy will refer to the same objects.

Understanding Deep Copy in C#:

In the case of deep copy, it will create a new object from the existing object and then copy the fields of the current object to the newly created object. If the field is a value type, then a bit-by-bit copy of the field will be performed. A new copy of the referred object will be created if the field is a reference type.

- **Value Types:** Like with shallow copies, value-type fields are copied directly.
- **Reference Types:** Instead of copying just the reference, a new copy of the referred object is created. Each reference in the original object leads to the creation of a new object in the copy.

Please have a look at the following image for a better understanding. This is the same example that we discussed with the Shallow Copy. But here, you can see that we have implemented the GetClone method in both the Employee and Address classes. From the Employee class GetClone method, we call the Address class GetClone method. This is how we can implement Deep Copy in C#.

```

public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public Address EmpAddress { get; set; }

    public Employee GetClone()
    {
        Employee employee = (Employee)this.MemberwiseClone();
        employee.EmpAddress = EmpAddress.GetClone();
        return employee;
    }
}

public class Address
{
    public string address { get; set; }
    public Address GetClone()
    {
        return (Address)this.MemberwiseClone();
    }
}

```

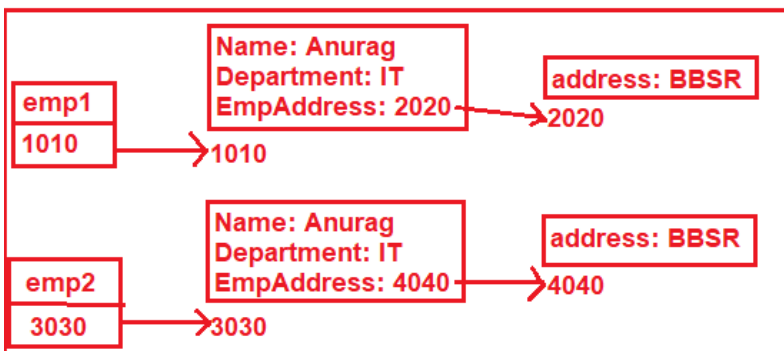
```

static void Main(string[] args)
{
    Employee emp1 = new Employee();
    emp1.Name = "Anurag";
    emp1.Department = "IT";
    emp1.EmpAddress = new Address()
    {
        address = "BBSR"
    };

    Employee emp2 = emp1.GetClone();
    emp2.Name = "Pranaya";
    emp2.EmpAddress.address = "Mumbai";
}

```

Client Code



Memory Representation

As shown in the above image, the Name and Department properties are value types, so it creates a copy of that and stores it in a different location. The EmpAddress is a Reference type property, and in Deep Copy, there is a clone of the reference type field, which will be stored in a different location. So, you need to remember that in the case of Deep Copy, the field type does not matter whether it is a value or a reference type. It always makes a copy of all the data and stores it in a different memory location.

Example to Understand Deep Copy in C#:

The following Example shows the use of Deep Copy in C#. The following Example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace DeepCopyDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Example to Understand Deep Copy
            //Creating Employee Object
            Employee emp1 = new Employee
            {
                Name = "Anurag",
                Department = "IT",
                EmpAddress = new Address() { address = "BBSR" }
            };
            //Creating a Clone Object from the Existing Object

```

```

        Employee emp2 = emp1.GetClone();
        //Changing Name Property of Clone Object Will Not Reflect the
Existing Object
        emp2.Name = "Pranaya";
        //Changing Address Property of Clone Object Will Not Reflect
the Changes in the Existing Object
        //This is because of Deep Copy
        //Both Clone and Existing Object have Memory locations for the
Address object
        emp2.EmpAddress.address = "Mumbai";
        Console.WriteLine("Employee 1: ");
        Console.WriteLine("Name: " + emp1.Name + ", Address: " +
emp1.EmpAddress.address + ", Dept: " + emp1.Department);
        Console.WriteLine("Employee 2: ");
        Console.WriteLine("Name: " + emp2.Name + ", Address: " +
emp2.EmpAddress.address + ", Dept: " + emp2.Department);
        Console.Read();
    }
}
public class Employee
{
    //Value type Property
    public string Name { get; set; }
    //Value type Property
    public string Department { get; set; }
    //Reference type Property
    public Address EmpAddress { get; set; }
    //Creating a Cloned Object of the Current Object
    public Employee GetClone()
    {
        Employee employee = (Employee)this.MemberwiseClone();
        //The following Statement will make this a Deep Copy
        //Now, Cloned and Existing Object have different Memory
Locations for the Address Object
        employee.EmpAddress = EmpAddress.GetClone();
        return employee;
    }
}
public class Address
{
    public string address { get; set; }
    //Creating a Cloned Object of the Current Object
    public Address GetClone()
    {
        return (Address)this.MemberwiseClone();
    }
}
}

```

Output:

```
Employee 1:  
Name: Anurag, Address: BBSR, Dept: IT  
Employee 2:  
Name: Pranaya, Address: Mumbai, Dept: IT
```

Note: In C#, a “deep copy” means creating a new object with a copy of the original object and all the objects inside it. In this case, changes to the new object won’t affect the original object or vice versa. To perform a deep copy, you typically have to manually copy each field of the object, taking care to also deep copy any reference types.

Differences Between Shallow Copy and Deep Copy in C#:

Shallow Copy and Deep Copy create a Clone Object from an Existing Object. The difference between them is that, in the case of Shallow copy, it will create the clone object from the existing object and then copy the value type fields of the existing object to the new object. But for the reference type property, it will only copy the reference, not the actual object itself. Therefore, the existing and cloned objects refer to the same memory location in the case of reference type.

On the other hand, in the case of Deep Copy, it will create the clone object from the existing object and then copy the fields of the existing object to the newly created clone object. If the field is a value type, then a bit-by-bit copy of the field will be performed. A new copy of the referred object will be created if the field is a reference type. Unlike the Shallow Copy, in the case of Deep Copy, both existing and clone objects have different memory locations for the reference type properties.

So, according to the application requirement, we need to decide whether to use Shallow Copy or Deep Copy. We can declare two methods within the class: one for Shallow Copy and another for Deep Copy. For a better understanding, please have a look at the following example. Here, within the Employee class, we have two methods. The GetShallowCopy method will make a Shallow Copy of the clone object, and the GetDeepCopy method will make a deep copy of the clone object.

```
using System;  
namespace ShallowCopyDeepCopyDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Example to Understand Deep Copy and Shallow Copy in C#  
            //Creating Employee Object  
            Employee emp1 = new Employee  
            {  
                Name = "Anurag",  
                Department = "IT",  
                EmpAddress = new Address() { address = "BBSR" }  
            };  
            //Creating a Clone Object from the Existing Object using the  
            GetDeepCopy method  
            Employee emp2 = emp1.GetDeepCopy();  
            //Changing Name and Address Property of Clone Object  
            emp2.Name = "Pranaya";  
            emp2.EmpAddress.address = "Mumbai";  
            Console.WriteLine("Using Deep Copy");  
            Console.WriteLine($" Employee 1: Name: {emp1.Name}, Address:  
{emp1.EmpAddress.address}");  
            Console.WriteLine($" Employee 2: Name: {emp2.Name}, Address:  
{emp2.EmpAddress.address}");  
            //Creating Employee Object
```

```

Employee emp3 = new Employee
{
    Name = "Sambit",
    Department = "HR",
    EmpAddress = new Address() { address = "Delhi" }
};
//Creating a Clone Object from the Existing Object using the
GetShallowCopy method
Employee emp4 = emp3.GetShallowCopy();
//Changing Name and Address Property of Clone Object
emp4.Name = "Hina";
emp4.EmpAddress.address = "Hyderabad";
Console.WriteLine("\nUsing Shallow Copy");
Console.WriteLine($" Employee 3: Name: {emp3.Name}, Address:
{emp3.EmpAddress.address}");
Console.WriteLine($" Employee 4: Name: {emp4.Name}, Address:
{emp4.EmpAddress.address}");

Console.Read();
}
}
public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public Address EmpAddress { get; set; }
    //Creating a Clone Object of the Current Object using Shallow Copy
    public Employee GetShallowCopy()
    {
        Employee employee = (Employee)this.MemberwiseClone();
        return employee;
    }
    //Creating a Clone Object of the Current Object using Deep Copy
    public Employee GetDeepCopy()
    {
        Employee employee = (Employee)this.MemberwiseClone();
        employee.EmpAddress = EmpAddress.GetClone();
        return employee;
    }
}
public class Address
{
    public string address { get; set; }
    public Address GetClone()
    {
        return (Address)this.MemberwiseClone();
    }
}
}
}

```

Output:

Using Deep Copy

Employee 1: Name: Anurag, Address: BBSR

Employee 2: Name: Pranaya, Address: Mumbai

Using Shallow Copy

Employee 3: Name: Sambit, Address: Hyderabad

Employee 4: Name: Hina, Address: Hyderabad

Both shallow and deep copies in C# pertain to replicating an object, but they differ in the depth of that replication. Here's a detailed breakdown of their differences:

When to Use Each

- **Shallow Copy:** This is useful when you want a quick copy of the object without needing to duplicate the objects it references. It is also useful when you are sure that the referenced objects will not be modified or that these changes should be reflected in all copies.
- **Deep Copy:** This is used when you need complete independence between the copies, such as when working with complex objects where changes to the copy should not affect the original object or vice versa.

In the next article, I will discuss [Real-Time Examples of the Prototype Design Pattern in C#](#). In this article, I try to explain the **Shallow Copy and Deep Copy in C#** with Examples. I hope you enjoy this Shallow Copy and Deep Copy in C# article.